

Este documento descreve a utilização da linguagem Python e da framework MASPY na programação de agentes inteligentes.

Instalação de ferramentas

Para utilização dos exemplos descritos aqui recomendam-se as seguintes ferramentas (*em ambiente Linux*).

- Instalando o Python no Linux
<https://python.org.br/instalacao-linux/>
- Instalando o Python 3 no Windows
<https://python.org.br/instalacao-windows/>
- Documentação da linguagem Python.
<https://docs.python.org/3/>
- Documentação para instalação do Python e do editor VSCode.
<https://learn.microsoft.com/pt-br/training/modules/python-install-vscode/>

Sumário

1	Introdução à Programação de Agentes em Python	2
1.1	Programação em Python	2
1.2	Programação de Agentes usando Python	2
1.3	Exemplos	2
2	Framework MASPY	16
2.1	Instalação	17
2.2	Modelagem	18
2.3	Exemplos	18

1 Introdução à Programação de Agentes em Python

Nesta seção são apresentando alguns exemplos de programação de agentes em Python.

1.1 Programação em Python

Python é uma linguagem popular e de propósito variado, mas muito usada para ciência de dados, inteligência artificial, jogos e programação web. A programação básica em Python é relativamente simples e por isso é utilizada em larga escala. É possível encontrar inúmeras fontes na Internet para aprender programação em Python. Algumas estruturas básicas que vamos utilizar neste material são listadas a seguir.

- Tuplas.
- Dicionários.
- Funções.
- Comandos de controle.
- Funções pré definidas.
- Classes.

1.2 Programação de Agentes usando Python

A programação de agentes pode ser realizada em qualquer linguagem de programação usando estruturas relativamente comuns nas linguagens. Contudo, aqui utilizamos Python visto que será empregada a framework MASPY na sequência. A programação de agentes BDI utiliza linguagens específicas como Jason, Gwendolen, entre outras.

Independente da linguagem temos a definição de um modelo do sistema de agentes que será implementado, que faz uso da arquitetura BDI (*Belief Desire Intention*).

Para implementar um agente BDI em Python poderiam ser usadas inúmeras formas. Aqui utilizamos a seguinte estrutura, onde as crenças, planos e ações são definidas de forma organizada na implementação do agente.

1.3 Exemplos

Nesta seção são apresentados seis exemplos. Os quatro primeiros exemplos utilizam o cenário de uma máquina de café, enquanto os dois últimos usam o cenário de veículos autônomos.

O primeiro exemplo é ilustrado no Código 1.

A Fig. 1 apresenta o resultado da execução do agente.

```

1 #
2 # Agente-01: fazer cafe
3 #
4
5 # Crenças:
6 # tuplas com 2 valores:
7 # crenças = ("poh_cafe", "cafeteira")
8 crenças = ("graos", "maquina")
9 # crenças = ("graos", "cafeteira")

```

```

10
11 print("Crenças do agente: ")
12 print(crencas)
13
14
15 # Planos:
16 def tem_cafe(crencas):
17     if crencas[0] == 'poh_cafe' or crencas[0] == 'graos':
18         return True
19     else:
20         return False
21
22 def fazer_cafe(crencas):
23     if crencas[0] == 'poh_cafe' and crencas[1] == 'cafeteira':
24         print("Fazer cafe filtrado")
25     elif crencas[0] == 'graos' and crencas[1] == 'maquina':
26         print("Fazer cafe expresso")
27     else: print("Nao fazer cafe (crencas conflitantes!)")
28
29
30 # Execucao:
31 cafe = tem_cafe(crencas)
32
33 if (cafe == True):
34     fazer_cafe(crencas)
35 else:
36     print("Ir ao mercado comprar cafe")

```

Código 1: Exemplo 1

```

• (.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-01.py
Crenças do agente:
('poh_cafe', 'cafeteira')
Fazer café filtrado
• (.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-01.py
Crenças do agente:
('graos', 'maquina')
Fazer café expresso
○ (.venv) gleifer@hanumat:~/codes$ 

```

Figura 1: Execução do Exemplo 1 da Máquina de Café

O segundo exemplo é ilustrado no Código 2.

As Figuras 2 e 3 apresentam os resultados de duas execuções do agente.

```

1 #-----
2 # Agente-02: fazer cafe
3 # com ciclo, sem crenças iniciais
4 #-----
5
6 # Estrutura de Crenças (tuplas com 2 valores):
7 # (graos, maquina)
8 # (poh_cafe, cafeteira)
9
10 # Planos:
11 def tem_cafe(crencas):
12     if crencas[0] == 'poh_cafe' or crencas[0] == 'graos':
13         return True
14     else:
15         return False

```

```

16
17 def fazer_cafe(crencas):
18     if crencas[0] == 'poh_cafe' and crencas[1] == 'cafeteira':
19         print("Fazer cafe filtrado \n")
20     elif crencas[0] == 'graos' and crencas[1] == 'maquina':
21         print("Fazer cafe expresso \n")
22     else: print("Nao fazer cafe (crencas conflitantes!) \n")
23
24
25 # Execucao:
26 # agente "conectado" ao ambiente
27 agente = 'on'
28 while agente == 'on':
29     cr1 = input("crenca 1: ")
30     cr2 = input("crenca 2: ")
31     crencas = cr1, cr2
32
33     print("\n")
34
35     cafe = tem_cafe(crencas)
36
37     if (cafe == True):
38         fazer_cafe(crencas)
39     else:
40         print("\n Ir ao mercado comprar cafe")
41
42     agente = input("\n Agente ativo ('on')? ")

```

Código 2: Exemplo 2

```

• (.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-02.py
crença 1: graos
crença 2: maquina

Fazer café expresso

Agente ativo ('on')? on
crença 1: poh_cafe
crença 2: cafeteira

Fazer café filtrado

Agente ativo ('on')? off
• (.venv) gleifer@hanumat:~/codes$ █

```

Figura 2: Execução 1 do Exemplo 2 da Máquina de Café

```

• (.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-02.py
crença 1: graos
crença 2: cafeteira

Não fazer café (crenças conflitantes!)

Agente ativo ('on')? on
crença 1: graos
crença 2: maquina

Fazer café expresso

Agente ativo ('on')? off
• (.venv) gleifer@hanumat:~/codes$ █

```

Figura 3: Execução 2 do Exemplo 2 da Máquina de Café

O terceiro exemplo é ilustrado no Código 3.

A Fig. 4 apresenta o resultado da execução do agente.

```

1  #
2  # Agente-03: maquina de cafe expresso
3  # com estrutura de dicionario
4  #
5
6  agente = {"nome": "maquinaCafe", "temAgua": False, "temGraos": False, "
7  capacidade": 2, "preparada": False}
8  print(f"Crenças iniciais do agente: {agente['nome']}")
9  print(agente)
10
11 # Planos:
12
13 def preparar_maquina(): # verificar agua & graos
14     if (agente["temAgua"] == False):
15         print("Recarregar agua")
16         recarregar_maquina()
17     else: print("Maquina carregada")
18     if (agente["temGraos"] == False):
19         print("Comprar graos")
20         comprar_graos()
21     else: print("Maquina com graos")
22
23 def recarregar_maquina():
24     print("Maquina recarregada")
25     agente["temAgua"] = True
26
27 def comprar_graos():
28     print("Maquina com graos")
29     agente["temGraos"] = True
30
31 def fazer_cafe():
32     print(f"\n Crenças do agente (preparando o cafe): {agente['nome']}")
33     print(agente)
34     print("Cafe pronto!")
35     agente["preparada"] = False
36     agente["temAgua"] = False
37     agente["temGraos"] = False
38

```

```

39 def maquina_preparada():
40     if (agente["temAgua"] == True) and (agente["temGraos"] == True):
41         agente["preparada"] = True
42         return True
43     else: return False
44
45
46 # Execucao:
47 # PS: observe um pequeno erro na execucao do agente
48
49 flag1 = maquina_preparada()
50 if (flag1 == True):
51     fazer_cafe()
52 else:
53     preparar_maquina()
54     fazer_cafe()
55
56 print(f"\n Crenças finais do agente: {agente['nome']}")
57 print(agente)
58
59 #

```

Código 3: Exemplo 3

```

(.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-03.py
Crenças iniciais do agente: maquinaCafe
{'nome': 'maquinaCafe', 'temAgua': False, 'temGraos': False, 'capacidade': 2, 'preparada': False}
Recarregar água
Máquina recarregada
Comprar grãos
Máquina com grãos

Crenças do agente (preparando o café): maquinaCafe
{'nome': 'maquinaCafe', 'temAgua': True, 'temGraos': True, 'capacidade': 2, 'preparada': False}
Café pronto!

Crenças finais do agente: maquinaCafe
{'nome': 'maquinaCafe', 'temAgua': False, 'temGraos': False, 'capacidade': 2, 'preparada': False}
(.venv) gleifer@hanumat:~/codes$

```

Figura 4: Execução do Exemplo 3 da Máquina de Café

O quarto exemplo é ilustrado no Código 4.

A Fig. 5 apresenta o resultado da execução do agente.

```

1 #
2 # Agente-04: maquina de cafe expresso
3 # com estrutura de dicionario e ciclo
4 #
5
6 agente = {"nome": "maquinaCafe", "temAgua": False, "temGraos": False, "
7     capacidade": 2, "preparada": False}
8 print(f"Crenças iniciais do agente: {agente['nome']}")
9 print(agente)
10
11 # Planos:
12 def preparar_maquina(): # verificar agua & graos
13     if (agente["temAgua"] == False):
14         print("Recarregar agua")
15         recarregar_maquina()
16     else: print("Maquina carregada")
17     if (agente["temGraos"] == False):

```

```

17         print("Comprar graos")
18         comprar_graos()
19     else: print("Maquina com graos")
20
21 def recarregar_maquina():
22     print("Maquina recarregada")
23     agente["temAgua"] = True
24
25 def comprar_graos():
26     print("Maquina com graos")
27     agente["temGraos"] = True
28
29 def fazer_cafe():
30     print(f"\n Crenças do agente (preparando o cafe): {agente['nome']}")
31     print(agente)
32     print("Cafe pronto!")
33     agente["preparada"] = False
34     agente["temAgua"] = False
35     agente["temGraos"] = False
36     print(f"\n Crenças do agente (tendo preparado o cafe): {agente['nome']}")
37     print(agente)
38
39 def maquina_preparada():
40     if (agente["temAgua"] == True) and (agente["temGraos"] == True):
41         agente["preparada"] = True
42         return True
43     else: return False
44
45
46 # Execução:
47 # agente "conectado" ao ambiente
48 agente_ativo = 'on'
49 while agente_ativo == 'on':
50
51     flag1 = maquina_preparada()
52     if (flag1 == True):
53         fazer_cafe()
54     else:
55         preparar_maquina()
56
57     print("\n")
58
59     agente_ativo = input("Agente ativo ('on')? ")
60
61 print(f"\n Crenças finais do agente: {agente['nome']}")
62 print(agente)
63
64 #

```

Código 4: Exemplo 4

8

```

• (.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-04.py
Crenças iniciais do agente: maquinaCafe
{'nome': 'maquinaCafe', 'temAgua': False, 'temGraos': False, 'capacidade': 2, 'preparada': False}
Recarregar água
Máquina recarregada
Comprar grãos
Máquina com grãos

Agente ativo ('on')? on

Crenças do agente (preparando o café): maquinaCafe
{'nome': 'maquinaCafe', 'temAgua': True, 'temGraos': True, 'capacidade': 2, 'preparada': True}
Café pronto!

Crenças do agente (tendo preparado o café): maquinaCafe
{'nome': 'maquinaCafe', 'temAgua': False, 'temGraos': False, 'capacidade': 2, 'preparada': False}

Agente ativo ('on')? on
Recarregar água
Máquina recarregada
Comprar grãos
Máquina com grãos

Agente ativo ('on')? off

Crenças finais do agente: maquinaCafe
{'nome': 'maquinaCafe', 'temAgua': True, 'temGraos': True, 'capacidade': 2, 'preparada': False}
• (.venv) gleifer@hanumat:~/codes$ █

```

Figura 5: Execução do Exemplo 4 da Máquina de Café

Agora, o quinto e sexto exemplos utilizam como cenário veículos autônomos, onde inclusive existe um segundo agente no sistema.

O quinto exemplo é ilustrado no Código 5.

A Fig. 6 apresenta o resultado da execução do agente.

```

1  #
2  # Agente-05: VA (veiculo autonomo)
3  # com estrutura de dicionario e ciclo
4  #
5
6  import random
7
8  agente = {"nome": "VA", "existeObstaculo": False, "velocidade": 60, "sensor": "
          sim", "localizacao": "naVIA"}
9  print(f"Crenças iniciais do agente: {agente['nome']}")
10 print(agente)
11 # localizacao: "naVIA" | "noCruzamento"
12
13 # Planos:
14
15 def verifica_obst():
16     obst = random.choice([True, False])
17     print(obst)
18     return obst
19
20 def desviar():
21     verifica_velocidade()
22     if (agente["velocidade"] >= 60):
23         print("Reduzir velocidade!")
24         agente["velocidade"] = agente["velocidade"] - 10
25     else: print("Manter velocidade p/ manobra")
26     print("Realizar manobra de desvio")

```



```

27
28 def verifica_velocidade():
29     velocidade_atual = random.randint(40,80)
30     agente["velocidade"] = velocidade_atual
31     print(f"\n Velocidade: {agente['velocidade']}")
32
33 def conduzir():
34     verifica_velocidade()
35     if (agente["velocidade"] >= 70):
36         print("Reduzir velocidade!")
37         agente["velocidade"] = agente["velocidade"] - 15
38     else: print("Manter velocidade p/ conducao")
39     print("Seguir conduzindo na via")
40
41
42 # Execucao:
43 # sensor do VA deve estar ativado
44 sensor = agente["sensor"]
45 while sensor == 'sim':
46
47     flag = verifica_obst()
48     if (flag == True):
49         print("Obstaculo na via")
50         agente["existeObstaculo"] = True
51         desviar()
52     else:
53         agente["existeObstaculo"] = False
54     conduzir()
55     print("\n")
56     sensor = input("Seguir na via? (sim/nao) ")
57
58 agente["sensor"] = sensor
59 print(f"\n Crenças finais do agente: {agente['nome']}")
60 print(agente)
61
62 #

```

Código 5: Exemplo 5

```

1 • (.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-05.py
Crenças iniciais do agente: VA
{'nome': 'VA', 'existeObstaculo': False, 'velocidade': 60, 'sensor': 'sim', 'localizacao': 'naVIA'}
True
Obstáculo na via

  Velocidade: 58
Manter velocidade p/ manobra
Realizar manobra de desvio

  Velocidade: 78
Reduzir velocidade!
Seguir conduzindo na via

Seguir na via? (sim/nao) sim
True
Obstáculo na via

  Velocidade: 41
Manter velocidade p/ manobra
Realizar manobra de desvio

  Velocidade: 63
Manter velocidade p/ condução
Seguir conduzindo na via

Seguir na via? (sim/nao) sim
True
Obstáculo na via

  Velocidade: 72
Reduzir velocidade!
Realizar manobra de desvio

  Velocidade: 80
Reduzir velocidade!
Seguir conduzindo na via

Seguir na via? (sim/nao) nao

Crenças finais do agente: VA
{'nome': 'VA', 'existeObstaculo': True, 'velocidade': 65, 'sensor': 'nao', 'localizacao': 'naVIA'}
• (.venv) gleifer@hanumat:~/codes$ █

```

Figura 6: Execução do Exemplo 5 do Agente Veículo Autônomo

O sexto exemplo é ilustrado no Código 6.

As Figuras 7 e 8 apresentam os resultados das execuções dos agentes.

```

1 #-----
2 # Agente-06: dois agentes
3 # VA (Veiculo Autonomo)
4 # AE (Autoridade Externa)
5 #-----
6
7 import random
8
9 agente = {"nome": "VA", "existeObstaculo": False, "velocidade": 60, "sensor": "
    sim", "localizacao": "naVIA"}
10 print(f"Crenças iniciais do agente: {agente['nome']}")
11 print(agente)
12 # localizacao: "naVIA" | "noCruzamento"
13
14 agenteAE = {"nome": "AE", "msg_do_VA": "ok", "velocidadeVA": 00, "sensorVA":
    False, "localizacaoVA": "NI"}
15 # "msg_do_VA": "ok" | "problema"
16 # NI = Nao Informada

```

```

17
18 # Planos:
19 ## verifica_obst(); desviar(); conduzir();
20
21 def verifica_obstaculo():
22     obst = random.choice([True, False])
23     return obst
24
25 #-----
26
27 def desviar():
28     verifica_velocidade()
29     if (agente["velocidade"] >= 60):
30         print("Reduzir velocidade!")
31         agente["velocidade"] = agente["velocidade"] - 10
32     else: print("Manter velocidade p/ manobra")
33     manobra_executada = realizar_manobra()
34     return manobra_executada
35
36 def verifica_velocidade():
37     velocidade_atual = random.randint(40,80)
38     agente["velocidade"] = velocidade_atual
39     print(f"\n Velocidade: {agente['velocidade']}")
40
41 def realizar_manobra():
42     manobra = random.choice([True, False])
43     if (manobra == True):
44         print("Realizar manobra de desvio")
45         return True
46     else:
47         print("Acionar Autoridade Externa!")
48         acionar_AE()
49         return False
50
51 def acionar_AE():
52     agenteAE["msg_do_VA"] = "problema"
53     agenteAE["velocidadeVA"] = agente["velocidade"]
54     agenteAE["sensorVA"] = agente["sensor"]
55     agenteAE["localizacaoVA"] = agente["localizacao"]
56     print(agenteAE)
57
58 #-----
59
60 def conduzir():
61     verifica_velocidade()
62     if (agente["velocidade"] >= 70):
63         print("Reduzir velocidade!")
64         agente["velocidade"] = agente["velocidade"] - 15
65     else: print("Manter velocidade p/ conducao")
66     print("Seguir conduzindo na via")
67
68
69 # Execucao:
70 # sensor do VA deve estar ativado
71 sensor = agente["sensor"]
72 while sensor == 'sim':
73
74     obstaculo = verifica_obstaculo()
75     if (obstaculo == True):
76         print("Obstaculo na via")

```

```

77     agente["existeObstaculo"] = True
78     desvio_executado = desviar()
79     if (desvio_executado == True):
80         conduzir()
81         print("\n")
82         sensor = input("Seguir na via? (sim/nao) ")
83     else:
84         sensor = 'nao' # desativar sensor e aguardar agenteAE
85         print("Aguardando Autoridade Externa")
86     else:
87         agente["existeObstaculo"] = False
88         conduzir()
89         print("\n")
90         sensor = input("Seguir na via? (sim/nao) ")
91 agente["sensor"] = sensor
92 print(f"\n Crenças finais do agente: {agente['nome']}")
93 print(agente)
94
95 #

```

Código 6: Exemplo 6

```

/home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-06.py
(.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-06.py
Crenças iniciais do agente: VA
{'nome': 'VA', 'existeObstaculo': False, 'velocidade': 60, 'sensor': 'sim', 'localizacao': 'naVIA'}
True
Obstáculo na via

Velocidade: 67
Reduzir velocidade!
Acionar Autoridade Externa!
{'nome': 'AE', 'msg_do VA': 'problema', 'velocidadeVA': 57, 'sensorVA': 'sim', 'localizacaoVA': 'naVIA'}
Aguardando Autoridade Externa

Crenças finais do agente: VA
{'nome': 'VA', 'existeObstaculo': True, 'velocidade': 57, 'sensor': 'nao', 'localizacao': 'naVIA'}
(.venv) gleifer@hanumat:~/codes$

```

Figura 7: Execução 1 do Exemplo 6 dos Agentes Veículo Autônomo e Autoridade Externa

```

• (.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-06.py
Crenças iniciais do agente: VA
{'nome': 'VA', 'existeObstaculo': False, 'velocidade': 60, 'sensor': 'sim', 'localizacao': 'naVIA'}
Obstáculo na via

Velocidade: 67
Reduzir velocidade!
Realizar manobra de desvio

Velocidade: 56
Manter velocidade p/ condução
Seguir conduzindo na via

Seguir na via? (sim/nao) sim
Obstáculo na via

Velocidade: 42
Manter velocidade p/ manobra
Realizar manobra de desvio

Velocidade: 66
Manter velocidade p/ condução
Seguir conduzindo na via

Seguir na via? (sim/nao) sim
Obstáculo na via

Velocidade: 78
Reduzir velocidade!
Acionar Autoridade Externa!
{'nome': 'AE', 'msg_do_VA': 'problema', 'velocidadeVA': 68, 'sensorVA': 'sim', 'localizacaoVA': 'naVIA'}
Aguardando Autoridade Externa

Crenças finais do agente: VA
{'nome': 'VA', 'existeObstaculo': True, 'velocidade': 68, 'sensor': 'nao', 'localizacao': 'naVIA'}
• (.venv) gleifer@hanumat:~/codes$

```

Figura 8: Execução 2 do Exemplo 6 dos Agentes Veículo Autônomo e Autoridade Externa

O sétimo exemplo é ilustrado no Código 7.

A Figura 9 apresenta o resultado da execução de dois agentes.

```

1  #
2  # Agente-07: VA (veiculo autonomo)
3  # com classe e metodos
4  #
5
6  import random
7
8  # definicao da classe do agente veiculo autonomo
9  class AgenteVA:
10     # inicializacao da classe
11     def __init__(self, nome, obst, veloc, sensor):
12         self.nome = nome
13         self.obst = obst
14         self.veloc = veloc
15         self.sensor = sensor
16
17     # metodo p/ verificar se ha obstaculo
18     def verifica_obst(self):
19         obst = random.choice([True, False])
20         return obst
21
22     # metodo p/ fazer o desvio
23     def desviar(self, velocidade):
24         if (velocidade >= 60):
25             print('Reduzir velocidade')

```

```

26         velocidade = velocidade - 10
27     else:
28         print('Manter velocidade')
29         print('Realizar desvio')
30
31     # metodo p/ verificar velocidade do veiculo
32     def verifica_velocidade(self):
33         veloc_atual = random.randint(40,80)
34         return veloc_atual
35
36     # metodo p/ conduzir o veiculo
37     def conduzir(self, velocidade):
38         if (velocidade >= 70):
39             print('Reduzir velocidade!')
40             velocidade = velocidade - 15
41         else:
42             print('Manter velocidade p/ conduzir veiculo')
43         return velocidade
44         print('Seguir conduzindo veiculo')
45
46     # metodo p/ mostrar crenças do agente
47     def crenças(self):
48         print('*** Crenças atuais do agente ***')
49         print('* nome VA: ', self.nome)
50         print('* obstaculo? ', self.obst)
51         print('* velocidade: ', self.veloc)
52         print('* sensor ativado? ', self.sensor)
53
54
55     # execucao do sistema (dois agentes)
56     # crenças iniciais do agente (cria instacia do agente)
57     print('*** Primeiro veiculo ***')
58     nomeVA = 'waymo'
59     agent1 = AgenteVA(nomeVA, False, 60, 'sim')
60     teste = agent1.verifica_obst()
61     if (teste == True):
62         print("Obstaculo na via")
63         velocidade = agent1.verifica_velocidade()
64         agent1.desviar(velocidade)
65         agent1 = AgenteVA(nomeVA, True, velocidade, 'sim')
66     else:
67         print("Nao tem Obstaculo")
68         velocidade2 = agent1.verifica_velocidade()
69         agent1 = AgenteVA(nomeVA, False, velocidade2, 'sim')
70     velocidade3 = agent1.verifica_velocidade()
71     velocidade4 = agent1.conduzir(velocidade3)
72     agent1 = AgenteVA(nomeVA, False, velocidade4, 'sim')
73     # mostra crenças atuais do agente
74     agent1.crenças()
75
76
77     print('\n *** Segundo veiculo ***')
78     nomeVA = 'tesla'
79     # cria instancia de um segundo agente
80     agent2 = AgenteVA(nomeVA, True, 80, 'nao')
81     # altera atributos/crenças do agente
82     agent2 = AgenteVA(nomeVA, False, 80, 'sim')
83
84     velocidade = agent2.verifica_velocidade()
85     velocidade2 = agent2.conduzir(velocidade)

```

```

86
87 agent2 = AgenteVA(nomeVA, False, velocidade2, 'sim')
88 # mostra crenças atuais do agente
89 agent2.crenças()

```

Código 7: Exemplo 7

```

* sensor ativado?  sim
• (.venv) gleifer@hanumat:~/codes$ /home/gleife
*** Primeiro veiculo ***
Obstaculo na via
Manter velocidade
Realizar desvio
Manter velocidade p/ conduzir veiculo
*** Crenças atuais do agente ***
* nome VA:  waymo
* obstaculo?  False
* velocidade:  45
* sensor ativado?  sim

*** Segundo veiculo ***
Reduzir velocidade!
*** Crenças atuais do agente ***
* nome VA:  tesla
* obstaculo?  False
* velocidade:  59
* sensor ativado?  sim
• (.venv) gleifer@hanumat:~/codes$

```

Figura 9: Execução do Exemplo 7 dos Agentes Veículo Autônomo usando classe e métodos

2 Framework MASPYP

O framework MASPYP (*MultiAgent System for PYthon*) foi desenvolvido principalmente para viabilizar a implementação de agentes BDI usando uma linguagem de propósito geral como base (no caso, Python).

Algumas das principais características do framework são:

- Utilização de decoradores Python para definição dos planos BDI dos agentes.
- Estrutura dos planos seguindo padrão BDI com evento gatilho, contexto e corpo dos planos.
- Disponibilização de ações internas.
- Canais de Comunicação entre agentes seguindo padrão de mensagens KQML.
- Implementação baseada em classes Python.
- Classe de Ambiente.

Gramática Abstrata

Abaixo segue a representação da gramática do MASPYP usando uma notação similar a BNF. Elementos complementares da gramática são definidas nas tabelas adiante.

$$\langle maspy \rangle ::= \langle ag \rangle^+ \mid \langle ag \rangle^+ \langle env \rangle^+$$

$$\langle ag \rangle ::= \text{Name Belief}^* \text{Goal}^* \langle plan \rangle^*$$

$$\langle plan \rangle ::= \langle trigger \rangle \langle context \rangle \langle body \rangle$$

$$\langle trigger \rangle ::= \text{Event}$$

$$\langle context \rangle ::= (\text{Information})^*$$

$$\langle body \rangle ::= \text{Function}(\dots) (\langle ag\text{-internal-action} \rangle \mid \langle comm \rangle)^*$$

$$\langle comm \rangle ::= \underline{\text{send}}(\langle target \rangle, \langle directive\text{-list} \rangle, \text{Information}, \langle channel \rangle) \\ \mid \underline{\text{receive}}(\langle directive\text{-list} \rangle, \text{Information})$$

$$\langle target \rangle ::= \text{Name}^+ \mid \underline{\text{broadcast}}$$

$$\langle channel \rangle ::= \underline{\text{default}} \mid \underline{\text{channel-ID}}$$

$$\langle env \rangle ::= \text{Name Percept}^* \langle env\text{-plan} \rangle^*$$

$$\langle env\text{-plan} \rangle ::= \text{Function}(\dots) (\langle env\text{-internal-action} \rangle)^*$$

Action	Functionality
add(B G List[B G])	Add Arg to agent
rm(B G List[B G])	Remove Arg from agent
add_plan(P List[P])	Add Plan(s) to agent
rm_plan(P List[P])	Remove Plan(s) to agent
test(B G)	Test Arg in agent
has(B G P E)	Check if agent has Arg
get(B G P E, B G P E)	Retrieve Arg_1 considering Arg_2
wait(Timeout,E)	Suspend until Timeout and(or) Event
connect_to(Env)	Connect and Gain Access to Env
disconnect_from(Env)	Disconnect and Lose Access from Env
perceive(Env)	Manually perceive Env Environment
drop_intention(B G)	Stop one Intention w/ Arg
drop_event(B G)	Stop one Event w/ Arg
drop_desire(B G)	Stop one Intention and Event w/ Arg
drop_all_intentions(B G)	Stop all Intentions w/ Arg
drop_all_events(B G)	Stop all Events w/ Arg
drop_all_desires(B G)	Stop all Intentions and Events w/ Arg
start()	Manually start the agent reasoning
stop_cycle()	End Reasoning Cycle

Tabela 1: MASPYP Agent Internal Actions

Action	Functionality
create(Percept List[Percept])	Create one or more Percepts
change(Percept, NewArgs)	Change the Args of a Percept
delete(Percept List[Percept])	Delete one or more Percepts
get(Percept)	Retrieve similar Percept(s)

Tabela 2: MASPYP Environment Internal Actions

Directive	Functionality
tell/untell	Add/Remove Belief(s)
achieve/unachieve	Add/Remove Goal(s)
askOne/askAll	Request One/All Belief—Goal/(s)
askOneReply	Request Belief—Goal, wait Reply
askAllReply	Request Beliefs—Goal, wait Reply
tellHow/untellHow	Add/Remove Plan(s)
askHow	Request Plan

Tabela 3: MASPYP Message KQML Directives

2.1 Instalação

- Repositório no github do framework MASPYP, incluindo documentação.
<https://github.com/laca-is/MASPYP>

- 18 • MASPYPY: link do framework no repositório de projetos Python.
<https://pypi.org/project/maspy-ml/>
- Veja como instalar utilizando o pip. Note que antes pode ser necessário instalar o pip (*Python Package Index*).
`sudo apt install python3-pip`
 - Então, pode proceder com a instalação da MASPYPY
`pip install maspy-ml`
 - Obs.: caso utilize o VSCode, proceder com a instalação via pip no próprio terminal do VSCode.
- A Fig. 10 ilustra a instalação do framework.

```
gleifer@hanumat:~$ pip install maspy-ml
Defaulting to user installation because normal site-packages is not writeable
Collecting maspy-ml
  Downloading maspy_ml-0.1.1-py3-none-any.whl (22 kB)
Installing collected packages: maspy-ml
Successfully installed maspy-ml-0.1.1
gleifer@hanumat:~$
```

Figura 10: MASPYPY: instalação via pip

2.2 Modelagem

Para representar os exemplos podemos utilizar modelos que ilustrem os elementos principais de um SMA com agentes BDI, no caso crenças, objetivos e planos e ainda também descrever os ambientes com suas percepções e ações e o aspecto de comunicação entre os agentes.

A Fig. 11 ilustra a legenda utilizada nos modelos da seção de exemplos.

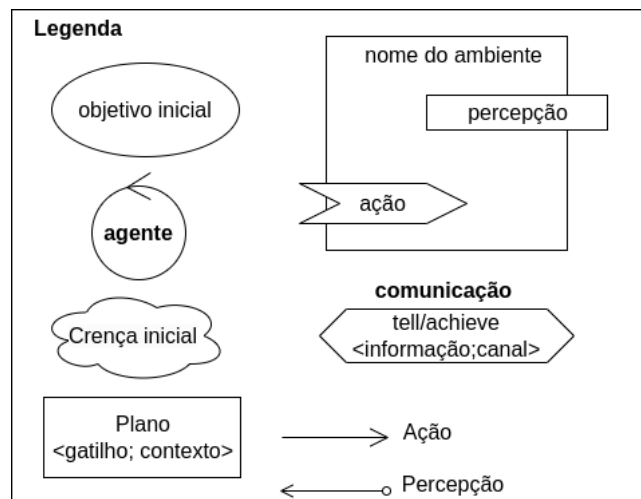


Figura 11: Legenda utilizada nos modelos

2.3 Exemplos

Seguem alguns exemplos de utilização do framework MASPYPY.

O primeiro exemplo é ilustrado no Código 8.

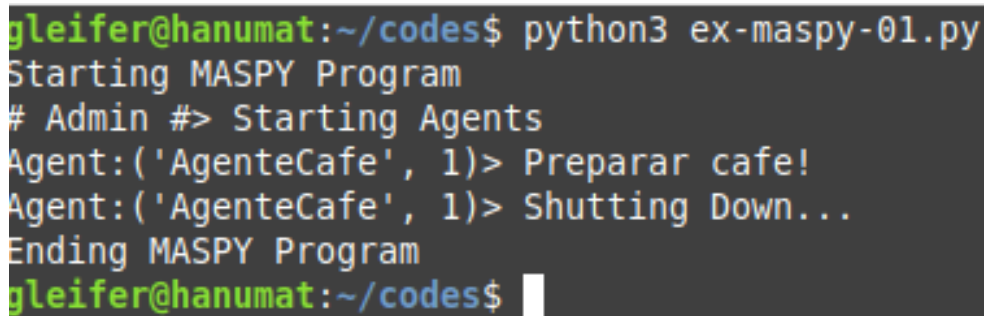
A Fig. 12 apresenta o resultado da execução do agente.

```

1  #-----
2  # Agente-MASPY-01: Preparar cafe
3  # criando um agente simples na MASPY
4  #-----
5
6
7  from maspy import *
8
9  class AgenteCafe (Agent):
10     @pl(gain, Belief("preparar_cafe")) # definicao do plano do agente
11     def fazer_cafe(self, src):
12         self.print("Preparar cafe!")
13         self.stop_cycle()
14
15 cafe = AgenteCafe() # instancia da classe do agente
16 cafe.add(Belief("preparar_cafe")) # adicao de crenca
17 Admin().start_system()

```

Código 8: Exemplo 1 (MASPY)



```

gleifer@hanumat:~/codes$ python3 ex-maspy-01.py
Starting MASPY Program
# Admin #> Starting Agents
Agent:('AgenteCafe', 1)> Preparar cafe!
Agent:('AgenteCafe', 1)> Shutting Down...
Ending MASPY Program
gleifer@hanumat:~/codes$

```

Figura 12: Execução do Exemplo 1 com o framework MASPY

O segundo exemplo é ilustrado no Código 9.

A Fig. 16 apresenta o resultado da execução do agente.

```

1  #-----
2  # Agente-MASPY-02: Maquina de Cafe
3  # criando um agente na MASPY com instancias distintas
4  #-----
5
6  from maspy import *
7
8  class MaquinaCafe (Agent):
9     def __init__(self, agent_name):
10         super().__init__(agent_name)
11         self.add(Belief("tem_agua")) # Crenca inicial
12
13     @pl(gain, Belief("tem_agua"), Belief("grao_cafe"))
14     def espresso_pronta(self, src):
15         self.print("Maquina Espresso pronta!")
16         self.add(Goal("fazer_cafe"))
17
18     @pl(gain, Goal("fazer_cafe"))
19     def preparar_cafe(self, src):
20         self.print("Preparar cafe!")
21         self.stop_cycle()

```

```

22
23     @pl(gain, Belief("tem_agua"), Belief("filtro"))
24     def cafeteira_pronta(self, src):
25         self.print("Cafeteira pronta!")
26         self.add(Goal("fazer_cafe"))
27
28     maquina1 = MaquinaCafe("Expresso")
29     maquina2 = MaquinaCafe("Cafeteira")
30     maquina1.add(Belief("grao_cafe"))      # adicao de crenca
31     maquina2.add(Belief("filtro"))        # adicao de crenca
32     Admin().start_system()

```

Código 9: Exemplo 2 (MASPY)

```

gleifer@hanumat:~/codes$ python3 ex-maspy-02.py
Starting MASPYP Program
# Admin #> Starting Agents
Agent:('Expresso', 1)> Maquina Expresso pronta!
Agent:('Expresso', 1)> Preparar cafe!
Agent:('Expresso', 1)> Shutting Down...
Agent:('Cafeteira', 1)> Cafeteira pronta!
Agent:('Cafeteira', 1)> Preparar cafe!
Agent:('Cafeteira', 1)> Shutting Down...
Ending MASPYP Program
gleifer@hanumat:~/codes$

```

Figura 13: Execução do Exemplo 2 com o framework MASPYP

Para o terceiro exemplo tem-se a modelagem (Fig. 14) e o Código 10. A Fig. 15 apresenta o resultado da execução do agente. Note que neste exemplo é usada a funcionalidade da classe `Admin` para geração de um relatório com detalhes da execução dos agentes, como informações de tempo, crenças, planos, etc.

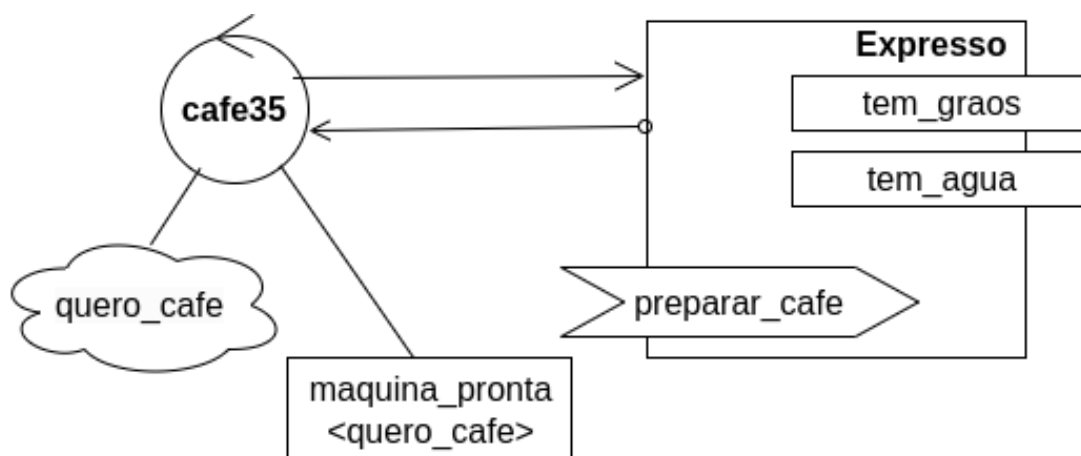


Figura 14: Modelagem do Exemplo 3 (MASPY)

```

1  #
2  # Agente-MASPYP-03: Maquina de Cafe como Ambiente do Agente
3  # interacao do ambiente e do agente
4  #

```

```

5
6 from maspy import *
7
8 class MaquinaCafe(Environment):
9     def __init__(self, env_name):
10         super().__init__(env_name)
11         self.create(Percept("tem_agua"))
12
13     def preparar_cafe(self, src):
14         self.print(f"Preparar cafe para o agente: {src}")
15
16
17 class AgenteCafe(Agent):
18     def __init__(self, agent_name):
19         super().__init__(agent_name)
20         self.add(Belief("quero_cafe")) # Crenca inicial
21
22     @pl(gain, Belief("quero_cafe"))
23     def maquina_pronta(self, src):
24         self.print("Verificar ambiente")
25         percepcao1 = self.get(Belief("tem_agua", source="Expresso"))
26         percepcao2 = self.get(Belief("tem_graos", source="Expresso"))
27         self.print(percepcao1.key) # imprime o valor das percepcoes
28         # obtidas do ambiente
29         self.print(percepcao2.key)
30         if percepcao1 and percepcao2: # testa se ha conteudo
31             nas "percepcoes"
32             self.preparar_cafe() # chama acao do ambiente
33             self.stop_cycle()
34
35 if __name__ == "__main__":
36     maquina = MaquinaCafe("Expresso")
37     agente = AgenteCafe("Cafe35")
38     Admin().connect_to([agente], [maquina])
39     maquina.create(Percept("tem_graos"))
40     Admin().full_report = True
41     Admin().start_system()

```

Código 10: Exemplo 3 (MASPY)

```

• (.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/py
Starting MASPYP Program
# Admin #> Starting Agents
Agent:Cafe35_1> Verificar ambiente
Agent:Cafe35_1> tem_agua
Agent:Cafe35_1> tem_graos
Environment:Expresso> Preparar cafe para o agente: Cafe35_1
# Admin #> [Closing System]
Ending MASPYP Program
Making System Report...
Managing Data...
Writing to Excel...
System Report Completed
○ (.venv) gleifer@hanumat:~/codes$ █

```

Figura 15: Execução do Exemplo 3 com o framework MASPYP

O quarto exemplo é ilustrado no Código 11.

A Fig. 16 apresenta o resultado da execução do agente.

```

1  #
2  # Agente-MASPYP-04: Veiculo Autonomo
3  # tenta realizar manobra de desvio de obstaculo
4  #
5
6  from maspy import *
7
8  import random
9
10 class Veiculo(Agent):
11     def __init__(self, agt_name):
12         super().__init__(agt_name)
13         self.add(Belief("obstaculo"))
14
15     @pl(gain, Goal("manobra"), Belief("obstaculo"))
16     def executar_manobra(self, src):
17         self.print("Realizar manobra")
18         manobra = random.choice([True, False])
19         if manobra == True:
20             self.print("Manobra realizada com sucesso")
21         else:
22             self.add(Goal("manobraCritica"))
23
24     @pl(gain, Goal("manobraCritica"), Belief("obstaculo"))
25     def informar_stakeholder(self, src):
26         self.print("Informar Autoridade Externa")
27         self.stop_cycle()
28
29 veiculo1 = Veiculo("Waymo")
30 veiculo1.add(Goal("manobra"))
31 Admin().start_system()

```

Código 11: Exemplo 4 (MASPY)

```

gleifer@hanumat:~/codes$ python3 ex-maspy-04.py
Starting MASPYP Program
# Admin #> Starting Agents
Agent:('Waymo', 1)> Realizar manobra
Agent:('Waymo', 1)> Manobra realizada com sucesso
^C# Admin #> [Closing System]
Agent:('Waymo', 1)> Shutting Down...
Ending MASPYP Program
gleifer@hanumat:~/codes$ python3 ex-maspy-04.py
Starting MASPYP Program
# Admin #> Starting Agents
Agent:('Waymo', 1)> Realizar manobra
Agent:('Waymo', 1)> Informar Autoridade Externa
Agent:('Waymo', 1)> Shutting Down...
Ending MASPYP Program
gleifer@hanumat:~/codes$ █

```

Figura 16: Execução do Exemplo 4 com o framework MASPYP

Para o quinto exemplo tem-se a modelagem (Fig. 17) e o Código 12. A Fig. 18 apresenta o resultado da execução do agente. Note que neste exemplo é usada a funcionalidade da classe Admin para geração de um relatório com detalhes da execução dos agentes, como informações de tempo, crenças, planos, etc.

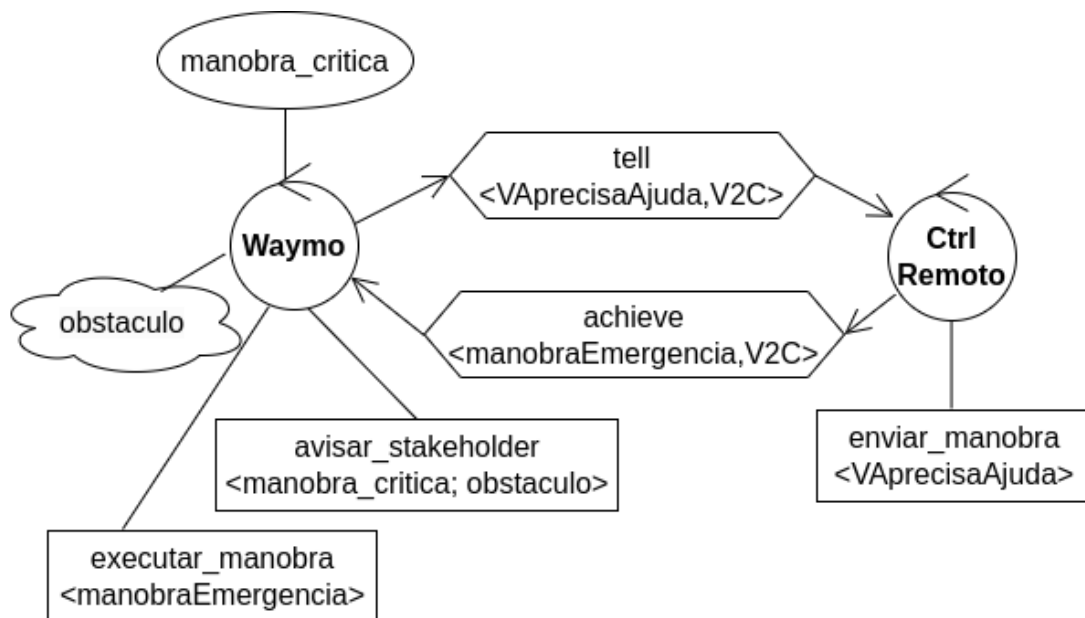


Figura 17: Modelagem do Exemplo 5 (MASPY)

```

1 #
2 # Agente-MASPYP-05: Veiculo Autonomo e Autoridade Externa (Stakeholder)
3 # comunicacao entre agentes
4 #
5
6 from maspy import *

```

```

7
8 class VA(Agent):
9     def __init__(self, agt_name):
10         super().__init__(agt_name)
11         self.add(Belief("obstaculo"))
12         self.add(Goal("manobraCritica"))
13
14     @pl(gain, Goal("manobraCritica"), Belief("obstaculo"))
15     def avisar_stakeholder(self, src):
16         # envia msg para o agente Stakeholder
17         self.print("Manobra critica; Acionar stakeholder")
18         self.send("ControladorRemoto", tell, Belief("VAprecisaAjuda"), "V2C")
19
20     @pl(gain, Goal("manobraEmergencia"))
21     def executar_manobra(self, src):
22         self.print(f"Manobra executada conforme stakeholder: {src}")
23         self.stop_cycle()
24
25 class Stakeholder(Agent):
26     def __init__(self, agt_name):
27         super().__init__(agt_name)
28
29     @pl(gain, Belief("VAprecisaAjuda"))
30     def enviar_manobra(self, src):
31         # envia msg para agente Waymo (VA)
32         self.send("Waymo", achieve, Goal("manobraEmergencia"), "V2C")
33         self.stop_cycle()
34
35 if __name__ == "__main__":
36     veiculo = VA("Waymo")
37     controlador = Stakeholder("ControladorRemoto")
38     comunicacao_ch = Channel("V2C") # V2C: Veiculo para Controlador
39     Admin().connect_to([veiculo, controlador], comunicacao_ch)
40     Admin().report = True
41     Admin().start_system()

```

Código 12: Exemplo 5 (MASPY)

```

(.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleife
Starting MASPYPY Program
# Admin #> Starting Agents
Agent:Waymo_1> Manobra critica; Acionar stakeholder
Agent:Waymo_1> Manobra executada conforme stakeholder: ControladorRemoto_1
# Admin #> [Closing System]
Ending MASPYPY Program
Making System Report...
Managing Data...
Writing to Excel...
System Report Completed
(.venv) gleifer@hanumat:~/codes$

```

Figura 18: Execução do Exemplo 5 com o framework MASPYPY

Para o sexto exemplo tem-se a modelagem (Fig. 19) e o Código 13. As Figuras 20 e 21 apresentam os resultados das execuções do agente.

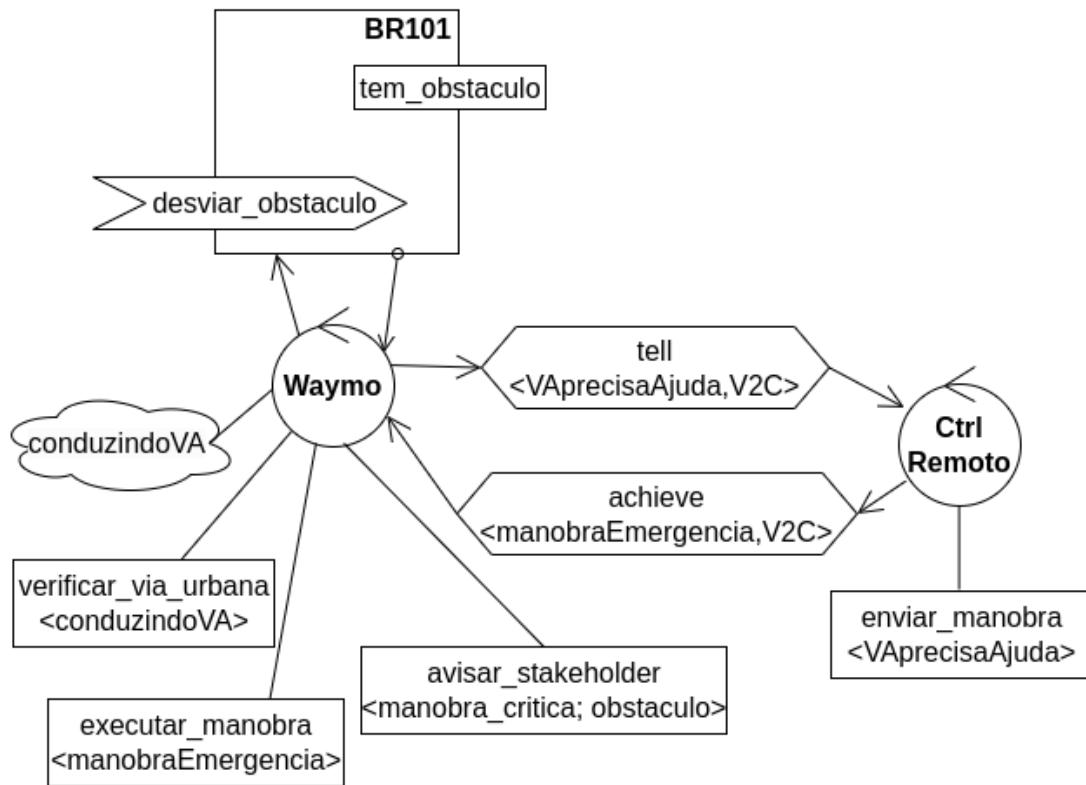


Figura 19: Modelagem do Exemplo 6 (MASPY)

```

1  #
2  # Agente-MASPY-06: VA, Autoridade Externa e Ambiente
3  # comunicacao entre agentes e interacao c/ ambiente
4  #
5
6  from maspy import *
7
8  import random
9
10 class MonitoramentoUrbano(Environment):
11     def __init__(self, env_name):
12         super().__init__(env_name)
13         self.create(Percept("tem_obstaculo"))
14
15     def desviar_obstaculo(self, src):
16         desvio = random.choice([True, False])
17         print(desvio)
18         if desvio == True:
19             self.print("Obstaculo desviado!")
20             self.create(Percept("obstaculo_resolvido"))
21         else:
22             self.print("Obstaculo nao desviado!")
23             self.create(Percept("obstaculo_nao_resolvido"))
24
25 class VA(Agent):
26     def __init__(self, agt_name):
27         super().__init__(agt_name)
28         self.add(Belief("conduzindoVA"))
29
30     @pl(gain, Belief("conduzindoVA"))
31     def verifica_via_urbana(self, src):
32         self.print("Verificar via urbana")

```

```

33     percepcao1 = self.get(Belief("tem_obstaculo",source="BR101"))
34     self.print(percepcao1.key)
35     if percepcao1:
36         self.desviar_obstaculo()
37
38     self.perceive("BR101")
39
40     percepcao2 = self.get(Belief("obstaculo_resolvido",source="BR101"))
41     if percepcao2:
42         self.print("Desvio realizado com sucesso!")
43         self.stop_cycle()
44     else:
45         percepcao3 = self.get(Belief("obstaculo_nao_resolvido",source="BR101
"))
46         if percepcao3:
47             self.add(Belief("obstaculo"))
48             self.add(Goal("manobraCritica"))
49
50     @pl(gain,Goal("manobraCritica"),Belief("obstaculo"))
51     def avisar_stakeholder(self,src):
52         # envia msg para o agente Stakeholder
53         self.print("Manobra critica; Acionar stakeholder")
54         self.send("ControladorRemoto",tell,Belief("VAprecisaAjuda"),"V2C")
55
56     @pl(gain,Goal("manobraEmergencia"))
57     def executar_manobra(self,src):
58         self.print(f"Manobra executada conforme stakeholder: {src}")
59         self.stop_cycle()
60
61     class Stakeholder(Agent):
62         def __init__(self, agt_name):
63             super().__init__(agt_name)
64
65         @pl(gain,Belief("VAprecisaAjuda"))
66         def enviar_manobra(self,src):
67             # envia msg para agente Waymo (VA)
68             self.send("Waymo",achieve,Goal("manobraEmergencia"),"V2C")
69             self.stop_cycle()
70
71     if __name__ == "__main__":
72         monitor = MonitoramentoUrbano("BR101")
73         veiculo = VA("Waymo")
74         controlador = Stakeholder("ControladorRemoto")
75         comunicacao_ch = Channel("V2C")
76         Admin().connect_to([veiculo,controlador],[comunicacao_ch,monitor])
77         Admin().start_system()

```

Código 13: Exemplo 6 (MASPY)

```

(.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-maspy-06.py
Starting MASPYP Program
# Admin #> Starting Agents
Agent:Waymo_1> Verificar via urbana
Agent:Waymo_1> tem_obstaculo
False
Environment:BR101> Obstaculo nao desviado!
Agent:Waymo_1> Does not contain Belief like Belief obstaculo_resolvido()[BR101]. Searched during verifica_via_urbana()
Agent:Waymo_1> Manobra critica; Acionar stakeholder
Agent:Waymo_1> Manobra executada conforme stakeholder: ControladorRemoto_1
# Admin #> [Closing System]
Ending MASPYP Program
(.venv) gleifer@hanumat:~/codes$

```

Figura 20: Execução 1 do Exemplo 6 com o framework MASPYP

```

(.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-maspy-06.py
Starting MASPYP Program
# Admin #> Starting Agents
Agent:Waymo_1> Verificar via urbana
Agent:Waymo_1> tem_obstaculo
True
Environment:BR101> Obstaculo desviado!
Agent:Waymo_1> Desvio realizado com sucesso!
^C# Admin #> [Closing System]
Still running agent(s):
ControladorRemoto_1 |
Ending MASPYP Program
(.venv) gleifer@hanumat:~/codes$

```

Figura 21: Execução 2 do Exemplo 6 com o framework MASPYP

O sétimo exemplo é ilustrado no Código 14.

As Figuras 22 e 23 apresentam os resultados das execuções do agente. Note que esse exemplo é similar ao anterior (Código 13), porém neste exemplo o ambiente cria novas percepções de forma dinâmica em tempo de execução. Com isso, o agente dispara um plano para tratar essas novas percepções obtidas do ambiente.

```

1  #
2  # Agente-MASPYP-07: VA, Autoridade Externa e Ambiente
3  # comunicacao entre agentes e interacao c/ ambiente
4  # cricao aleatoria de percepcoes no ambiente (tempo de execucao)
5  #
6
7  from maspy import *
8
9  import random
10
11 class MonitoramentoUrbano(Environment):
12     def __init__(self, env_name):
13         super().__init__(env_name)
14         self.create(Percept("tem_obstaculo"))
15
16     def desviar_obstaculo(self, src):
17         desvio = random.choice([True, False])
18         print(desvio)
19         if desvio == True:
20             self.print("Obstaculo desviado!")
21             self.create(Percept("obstaculo_resolvido"))
22         else:

```

```

23         self.print("Obstaculo nao desviado!")
24         self.create(Percept("obstaculo_nao_resolvido"))
25
26 class VA(Agent):
27     def __init__(self, agt_name):
28         super().__init__(agt_name)
29         self.add(Belief("conduzindoVA"))
30
31     @pl(gain, Belief("conduzindoVA"))
32     def verifica_via_urbana(self, src):
33         self.print("Verificar via urbana")
34         percepcao1 = self.get(Belief("tem_obstaculo", source="BR101"))
35         self.print(percepcao1.key)
36         if percepcao1:
37             self.desviar_obstaculo()
38             self.perceive("BR101")
39             self.add(Goal("verificar_acao"))
40
41     @pl(gain, Goal("verificar_acao"), Belief("conduzindoVA"))
42     def verificar_acao(self, src):
43         percepcao2 = self.get(Belief("obstaculo_resolvido", source="BR101"))
44         percepcao3 = self.get(Belief("obstaculo_nao_resolvido", source="BR101"))
45         if percepcao2:
46             self.print("Desvio realizado com sucesso!")
47             self.send("ControladorRemoto", tell, Belief("VA_operacional"))
48             self.stop_cycle()
49         elif percepcao3:
50             self.add(Belief("obstaculo"))
51             self.add(Goal("manobraCritica"))
52         else:
53             self.add(Goal("verificar_acao"))
54
55     @pl(gain, Goal("manobraCritica"), Belief("obstaculo"))
56     def avisar_stakeholder(self, src):
57         # envia msg para o agente Stakeholder
58         self.print("Manobra critica; Acionar stakeholder")
59         self.send("ControladorRemoto", tell, Belief("VA_precisa_ajuda"), "V2C")
60
61     @pl(gain, Goal("manobraEmergencia"))
62     def executar_manobra(self, src):
63         self.print(f"Manobra executada conforme stakeholder: {src}")
64         self.stop_cycle()
65
66 class Stakeholder(Agent):
67     def __init__(self, agt_name):
68         super().__init__(agt_name)
69
70     @pl(gain, Belief("VA_precisa_ajuda"))
71     def enviar_manobra(self, src):
72         # envia msg para agente Waymo (VA)
73         self.send("Waymo", achieve, Goal("manobraEmergencia"), "V2C")
74         self.stop_cycle()
75
76     @pl(gain, Belief("VA_operacional"))
77     def desligar(self, src):
78         self.stop_cycle()
79
80 if __name__ == "__main__":
81     monitor = MonitoramentoUrbano("BR101")
82     veiculo = VA("Waymo")

```

```

83     controlador = Stakeholder("ControladorRemoto")
84     comunicacao_ch = Channel("V2C")
85     Admin().connect_to([veiculo, controlador], [comunicacao_ch, monitor])
86     Admin().start_system()

```

Código 14: Exemplo 7 (MASPY)

```

(.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-maspy-07.py
Starting MASPYP Program
# Admin #> Starting Agents
Agent:Waymo_1> Verificar via urbana
Agent:Waymo_1> tem_obstaculo
True
Environment:BR101> Obstaculo desviado!
Agent:Waymo_1> Does not contain Belief like Belief obstaculo_nao_resolvido()[BR101]. Searched during verificar_acao()
Agent:Waymo_1> Desvio realizado com sucesso!
# Admin #> [Closing System]
Ending MASPYP Program
(.venv) gleifer@hanumat:~/codes$

```

Figura 22: Execução 1 do Exemplo 7 com o framework MASPYP

```

(.venv) gleifer@hanumat:~/codes$ /home/gleifer/codes/.venv/bin/python /home/gleifer/codes/ex-maspy-07.py
Starting MASPYP Program
# Admin #> Starting Agents
Agent:Waymo_1> Verificar via urbana
Agent:Waymo_1> tem_obstaculo
False
Environment:BR101> Obstaculo nao desviado!
Agent:Waymo_1> Does not contain Belief like Belief obstaculo_resolvido()[BR101]. Searched during verificar_acao()
Agent:Waymo_1> Manobra critica; Acionar stakeholder
Agent:Waymo_1> Manobra executada conforme stakeholder: ControladorRemoto_1
# Admin #> [Closing System]
Ending MASPYP Program
(.venv) gleifer@hanumat:~/codes$

```

Figura 23: Execução 2 do Exemplo 7 com o framework MASPYP